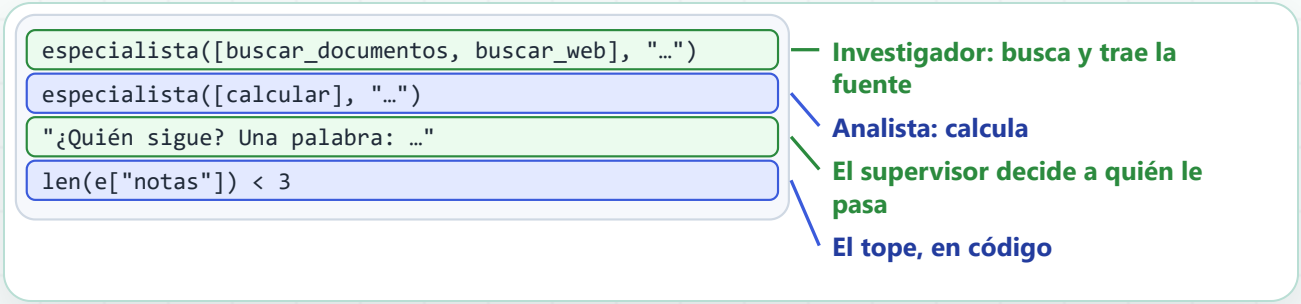
Taller: escribí y dibujá tu grafo (60 min)

1. **Escribí** `v05_equipo.py`
2. **Dibujo** en mermaid.live.
3. **Notas** de cada uno.

Práctica M7, arranque y pasos 1 a 3.

Primer control · 10–15 min: grafo, borrador y dos respuestas.



La persona que aprueba es el último control

Borrador del modelo chico
«la tabla tiene 15 filas; ×3 = 45»



Rechazo
sin respuesta final

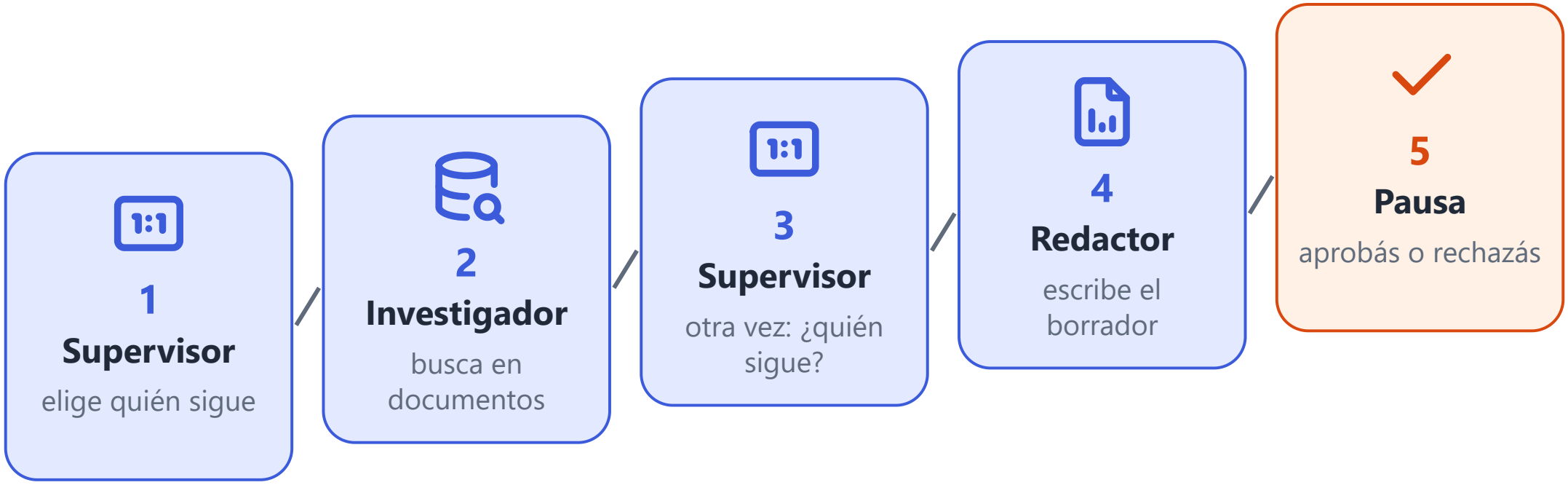
- La tabla real tiene 5 filas, no 15: el borrador está mal.
- Ni supervisor ni redactor lo vieron. ¿Quién lo detecta?

Con Gemini acertó, pero salteó al analista

```
[supervisor]   {'siguiente': 'investigador', 'pasos': 1}
[investigador] {'notas': ['Según la tabla comparativa entre Checkpointer y Store…
[supervisor]   {'siguiente': 'redactor', 'pasos': 2}
[redactor]     {'borrador': 'Según la documentación, la tabla … tiene 5 filas…
[PAUSA]        espera a una persona · borrador:
               … tiene 5 filas de contenido. Multiplicando ese número por 3,
               el resultado es 15.
[aprobado]     respuesta final: '… el resultado es 15.'
[rechazo]      respuesta final: (ninguna: el rechazo la bloqueó)
```

La cuenta la hizo el redactor, sin calculadora: salió bien, pero nadie la comprobó.

La ruta que tomó: el analista no se llamó



El supervisor decide la ruta en cada consulta: esta vez, sin analista.

¿Cómo harías que pida al analista cuando hay una cuenta?

El grafo se pausa y retoma

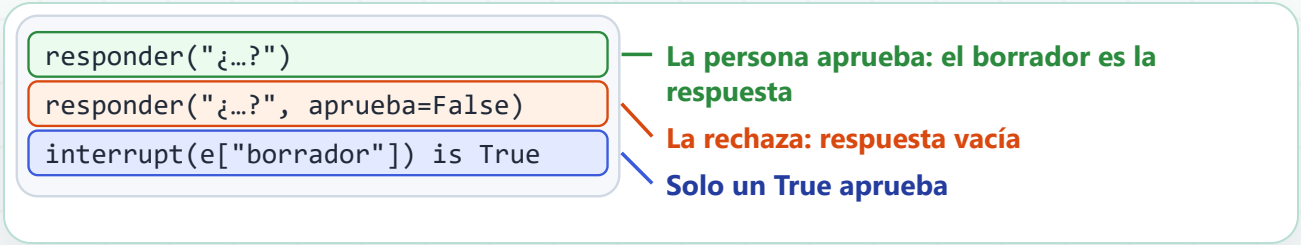
```
def aprobar(e):  # PAUSA hasta que una persona decide
    return {"aprobada": interrupt(e["borrador"]) is True}

grafo.invoke({"pregunta": pregunta, "notas": []}, hilo)  # hasta aprobar
e = grafo.invoke(Command(resume=True), hilo)  # mismo hilo, la decisión
```

Como una aprobación de Power Automate: el flujo espera a que alguien firme.

Taller: aprobar, rechazar y comparar (40 min)

1. **Aprobar y rechazar:**
   `aprueba=True` / `False`
2. **Rechazo** = sin respuesta final.
3. **v0.4 vs v0.5**, misma suite.



Práctica M7, «En clase», pasos 4 y 5.

Trampa: ¿el rechazo responde? Mirá `responder`.

Paso 5 de 5: Comparar

¿Qué cambiarías en el supervisor?

¿El modelo?

uno más grande

¿Las instrucciones?

lo que dice SUPERVISOR

¿El tope de pasos?

MAX_PASOS

Elegí una y decí por qué.

El supervisor del modelo chico mandó preguntas de web al analista.

CrewAI piensa en roles, LangGraph, en estado

	CrewAI	LangGraph
Se arma con	Roles	Estado y nodos
Quién decide el orden	El proceso: secuencial o jerárquico	Vos, con aristas
Pausa humana	`human_input=True` en una tarea	`interrupt` en un nodo

Repaso: hoy te llevás tres ideas



Equipo

un supervisor reparte



Grafo

cajas, flechas y estado



Aprobación

una persona aprueba al final

Más agentes no siempre es mejor: medí el costo antes de sumarlos.

¿Qué pasa del investigador al redactor?

Clínica: mostrá tu tabla «agente solo vs equipo» y la prueba de rechazo.

¿Qué información pasa del investigador al redactor, y por qué esa y no toda?

Anotá tu **confianza (1 a 4)**.

Producción

M8: v1.0, Producción



Hoy: el recorrido

De «anda en mi máquina» a «lo liberaría»



Venís de v0.5: un equipo con aprobación humana. Hoy juntás la evidencia para liberarlo.

Hoy cada ejemplo corre en vivo: cuaderno de la clase 7.

Sin mirar: ¿qué no podrías explicar?



?

Si falla mañana

con un usuario real, ¿qué parte no podrías explicar?

Anotala: es lo primero que hay que poder ver.

¿Qué parte no podrías explicar?

Respuestas: traza, costo y guardrail

Traza

qué herramienta usó y con qué datos

Tokens y precio

cuánto costó esa consulta

Guardrail

por qué dijo lo que no debía, con su test

¿Para qué te sirve? Saber si lo podés soltar

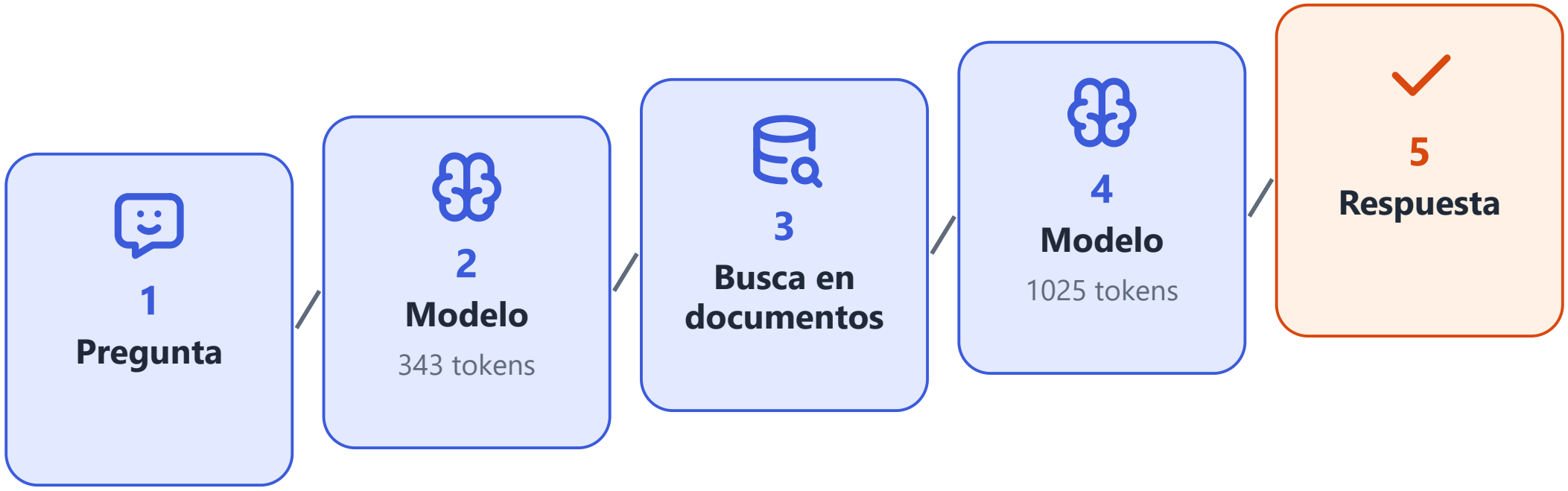
Tu jefe pregunta	Respondés con
«¿Por qué respondió eso?»	La traza
«¿Cuánto cuesta por mes?»	Tokens × precio
«¿Y si le piden datos de clientes?»	Un guardrail con test
«¿Ya lo podemos usar?»	Evidencia

Producción: el agente con sus controles



Lo usa otra persona: cada consulta deja traza y costo.

Una traza muestra cada paso de una consulta



Traza: el registro de cada paso, como la caja negra de un avión.

Como el Analizador de rendimiento de Power BI: cuánto tardó cada paso.

Cada pregunta deja una línea en la traza

```
{"pregunta": "¿Qué hace interrupt()? Respondeme a [email]",
 "precio_del": "19/10", "herramientas": ["buscar_documentos"],
 "llamadas": 2, "tokens_entrada": 1207, "tokens_salida": 161,
 "costo_usd": 0.000765, …}

por mes, a 1000 por día: US$ 23
```

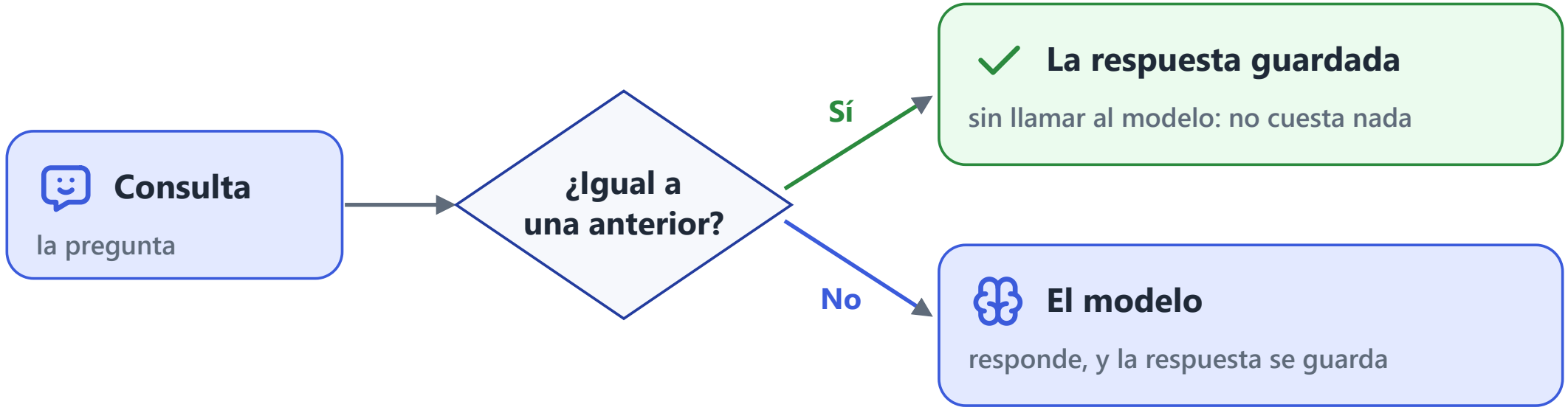
- El email no llegó: lo tapó el guardrail.
- Langfuse o LangSmith guardan además cada paso.

Del token al mes: multiplicá y proyectá

	Por consulta	1000 por día, 30 días
Tokens de entrada	1207	36 millones
Tokens de salida	161	4,8 millones
Costo*	US$ 0,000765	US$ 23

* Con US$ 0,30 y 2,50 por millón (19/10).

Una caché responde lo repetido sin el modelo



Riesgo: devolver algo viejo; y si compara por parecido, algo equivocado

La **semántica** compara por parecido: ahorra más, se equivoca más.

Repetida: 0 segundos. Con una minúscula: al modelo

```
set_llm_cache(InMemoryCache())    # guarda por el texto exacto del pedido

1.31 s · tokens 52 · «¿Qué hace interrupt()…»
0.00 s · tokens 52 · «¿Qué hace interrupt()…»    ← la guardada
0.84 s · tokens 54 · «¿qué hace interrupt()…»    ← distinta: otra llamada
```

- La guardada trae sus tokens viejos: si sumás el costo con esos números, contás de más.
- Probalo: cuaderno en vivo de la clase 7, sección 6

Un guardrail en código cambia la respuesta

Sin guardrail
se cae el modelo →
«ConnectionError: …»

→

Con guardrail
«No pude responder ahora.
Probá en unos minutos.»

- **Guardrail** = un control en código, como una baranda.
- Si responde de memoria sin buscar, no lo ataja: ¿qué test lo muestra?

Más guardrails que los de LangChain

Middleware de LangChain

NeMo Guardrails (NVIDIA)

Guardrails AI

Llama Guard (Meta)

Revisan lo que entra o sale; la **moderación** frena contenido dañino (Llama Guard). Ninguno ve lo que no previste.

Si algo se cae, avisa en vez de inventar

```
try:
    r = base.responder(tapar(pregunta))
except Exception as error:
    r = {"respuesta": "No pude responder ahora…"}
```

— Tu versión de siempre, sin emails

— Si algo se cae (modelo, red)…

— …un aviso claro, no un invento

Un aviso honesto, no un error en inglés.

El email se tapa antes de llegar al modelo

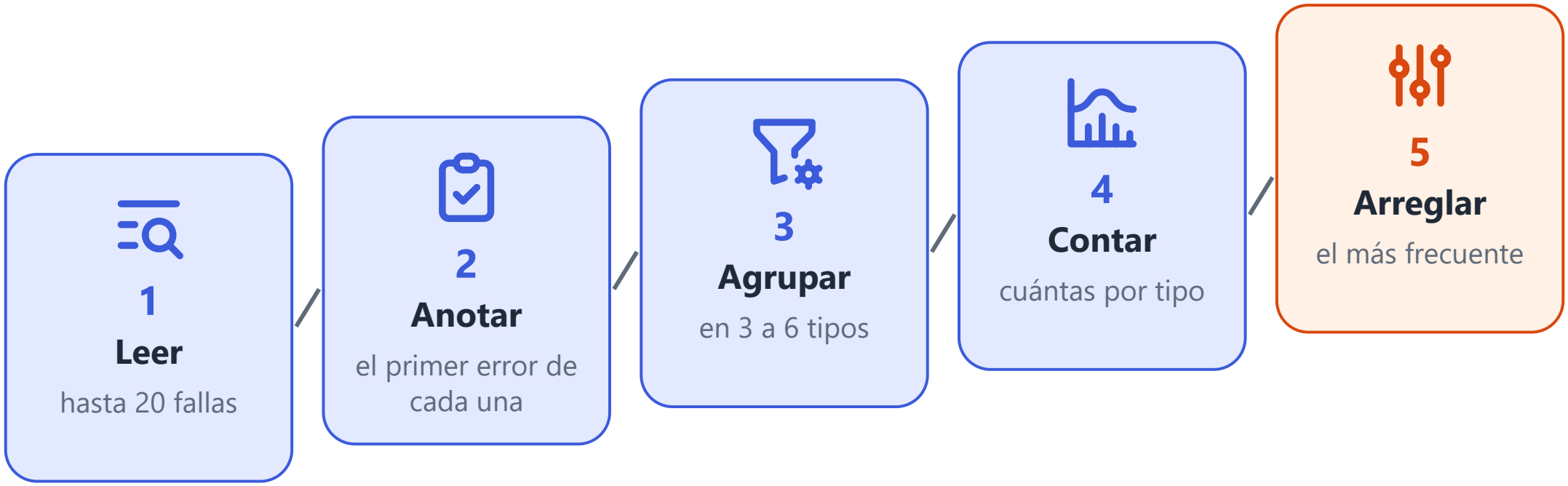
Escribe la persona
«Mi mail es ana.perez@ejemplo.com»

→

Ve el modelo
«Mi mail es [email]»

Sin consentimiento, no sale (Ley 18.331, de datos personales).

Antes de medir más, leé las respuestas que fallaron



Como mirar los residuos en ML antes de cambiar el modelo.

Cada tipo de error pide su propio control



Sigue una regla

se controla con código



Pide criterio

un modelo juez: sí o no



Juez nuevo

comparalo con 20 casos anotados

Sí o no: un puntaje de 1 a 10 no dice qué arreglar.

Modelo chico en v0.5: las 9 fallas, al elegir

```
CSV anotado (4 de las 9 filas)
id  pregunta (recortada)                    usó → esperaba                  primer error
1   ¿Qué hace interrupt() en LangGraph?      nada → buscar_documentos        no buscó
3   ¿Qué clase borra mensajes del estado?    calcular → buscar_documentos    equivocada
5   ¿Última versión de LangGraph en PyPI?    calcular ×3 → buscar_web        equivocada
11  Gracias, eso es todo.                    buscar_documentos → nada        de más
2.ª corrida: la cuenta
primer_error
no buscó en los documentos           4
pidió una herramienta equivocada     4
usó una herramienta sin necesidad    1
```

- Modelo local chico (3 de 12).
- Ninguna fue «buscó bien y respondió mal».

Análisis de errores: una tabla dinámica de tus fallas

```
logs/medir.csv      →  filtrá las preguntas que fallaron
                    →  sumá la columna primer_error: dónde se rompió primero
tabla dinámica      →  cuántas fallas de cada tipo
```

- En Excel o Power BI, como un análisis de causa raíz.
- Arreglá primero el tipo más frecuente.

Taller v1.0: traza, costo y tu peor fallo (25 min)

1. **Tu v1.0** y su traza.
2. **Tests:** el email y la caída.
3. **Costo** con `medir.py`, web grabada.

Práctica M8, arranque y «En clase», pasos 1 a 3.

Primer control · 10–15 min: en la traza, ningún email y el costo.

```
PRECIO_ENTRADA, PRECIO_SALIDA   — US$ por millón: de la página del proveedor
PRECIO_FECHA = "19/10"          — El día que los miraste
def tapar(texto):               — El guardrail: un email no pasa
open("logs/traza.jsonl", "a")   — Una línea por pregunta
```

Repaso: hoy te llevás tres ideas



Traza

ver cada paso y sus tokens



Costo

tokens por precio, al mes



Guardrail

un control en código

Antes de medir más, leé las respuestas que fallaron.

¿Qué te impide liberarlo hoy?

Nombrá lo que hoy te frena y la prueba que lo resolvería.

- Anotá tu **confianza (1 a 4)**.
- Entregás **hasta el dom 22/11, 23:59**; la lista, en la tarea «Entrega final» del aula virtual.